



Audit Boutique

Panther Staking Smart Contract Audit Report II

Produced for:

Panther Ventures Limited

Produced by:

Dominik Spicher, Audit Boutique

6 December 2022



Contents

1. Executive Summary	3
2. Scope	3
3. Methodology	3
4. Best practices checklist	3
5. Findings	4
5.1 High severity	4
5.2 Medium severity	4
5.3 Low severity	4
5.3.1 Delay variable protecting against frontrunning attacks starts out at a dangerous zero value	4
5.3.2 Obscuring arithmetic in calculating updated reward limits	5
5.4 Comments	6
6. Limitations	6



1. Executive Summary

The audit has uncovered no issues of high severity, no issues of medium severity, and three issues of low severity.

2. Scope

The review was performed on Git revision 6dacb91a373d6fe2088533b7782a249eb170a78a of the panther-core repository.

In particular, the review pertained to the following smart contract source files, specified here with their sha256 prefixes:

contracts/contracts/staking/AdvancedStakeRewardAdviserAndMsgSender.sol	663eb8afcf3038c37b12
contracts/contracts/staking/AdvancedStakeActionMsgRelayer.sol	9248512b7c846d1651c6
contracts/contracts/staking/AdvancedStakeRewardController.sol	45616c35fc3061142475
contracts/contracts/staking/PNftToken.sol	0d51c3f72310d12c8b0f
contracts/contracts/protocol/PantherPoolV0.sol	37e36f3c075a66542253
contracts/contracts/protocol/Vault.sol	7eac2a87830554e8691e
contracts/contracts/protocol/ZAssetsRegistry.sol	5c24f60a8dd528a2142a

Imported contracts from well-known libraries such as Open-Zeppelin were not part of the review.

The review was constrained to the Solidity source file. Low-level assembly code generated thereof was not inspected.

3. Methodology

The review consisted of the following steps:

- Check for compliance with smart contract development best practices
- Check for compliance with specification, where available and applicable
- Manual inspection and analysis of the smart contract and test code
- Usage of static analysis tools

4. Best practices checklist

Non-adherence to the characteristics listed in the below list does not represent a security issue itself. However, following best practices is a good indicator of care and attention to detail, it allows the review process to be more focused and efficient, thus making it more likely for issues to be uncovered.

- ✓ The code was provided as a Git repository
- ✓ There exists specification, covering the most important aspects of smart contract functionality
- ✓ The development process is understandable through well-delineated, atomic commits
- ✓ Code duplication is minimal
- ✓ The smart contract source files are provided in an unflattened manner
- ✓ The code compiles with a recent Solidity version



- ✓ The code is consistently formatted
- ✓ Code comments are in line with the associated code (mostly)
- ✓ There are tests
- ✓ Tests are easy to run
- ✓ Tests provide a reasonably high coverage (mostly, see Appendix A)
- ✓ The code is well documented
- ✓ There is no commented code
- ✓ There is no unused code
- The naming scheme is consistent

Commendably, the project's Continuous Integration pipeline also includes the slither static analyzer,¹ significantly increasing the assurances derivable from a successful build.

5. Findings

We rank our findings according to their perceived severity (high, medium, low), where an issue's severity is understood to be the product of its likelihood of being triggered and the impact of its consequences. Where deemed appropriate, we also report issues that are not security-relevant per-se but impact things like transaction-cost effectiveness or user experience.

5.1 High severity

No issues were found.

5.2 Medium severity

No issues were found.

5.3 Low severity

5.3.1 Delay variable protecting against frontrunning attacks starts out at a dangerous zero value

Status: Acknowledged

Panther Response: This variable is intentionally uninitialized to integrate with the proxy pattern.

In `PantherPoolV0.sol`, the `exitDelay` variable protects against frontrunning attacks when users exit the system. To do so, it must be set to a non-zero value. Indeed, the `updateExitTimes` function protects against setting a zero value. However, `exitDelay` is not initialized in the constructor and therefore starts out in its uninitialized zero state. This presents a potential risk if the commit-enter-exit functionality is inadvertently used before the exit delay has been properly initialized.

¹ <https://github.com/crytic/slither>



5.3.2 Obscuring arithmetic in calculating updated reward limits

In `AdvancedStakeRewardController.sol`, the `_getUpdatedLimit()` function calculates updated reward limits based on some input values. Ignoring types (and therefore intermediate under- and overflows) for the moment, the variables are computed in the following manner (we have converted a ternary expression to an if-else to increase readability):

```
unusedLimit = currentLimit - usedLimit;
if (available > unusedLimit) {
    newAllocation = available - unusedLimit;
    limit = newAllocation + currentLimit;
} else {
    shortage = unusedLimit - available;
    if (currentLimit > shortage) {
        limit = currentLimit - shortage;
    } else {
        limit = 0;
    }
}
```

By putting in definitions, this can easily be shown to be arithmetically identical (again, ignoring Solidity type conversions and over- and underflows) to

```
if (available > currentLimit - usedLimit) {
    limit = (available - (currentLimit - usedLimit)) + currentLimit;
} else {
    if (currentLimit > (currentLimit - usedLimit) - available) {
        limit = currentLimit - ((currentLimit - usedLimit) - available);
    } else {
        limit = 0;
    }
}
```

and therefore to

```
if (available > currentLimit - usedLimit) {
    limit = available + usedLimit;
} else {
    if (currentLimit > currentLimit - usedLimit - available) {
        limit = available + usedLimit;
    } else {
        limit = 0;
    }
}
```

Finally, it can be seen that the limit variable is computed identically in the two main arms, and that the second else-condition (where a zero limit is set) is never entered into. Both of these facts indicate that the developer intended for the code to behave differently, or did not anticipate the simpler description.

Note that the simpler description is consistent with the early exit of returning `currentLimit` on L570, as



```
available + usedLimit == unusedLimit + usedLimit == currentLimit
```

```
if available == unusedLimit.
```

5.4 Comments

No issues found.

6. Limitations

Even though the code has been reviewed carefully on a best-effort basis, undiscovered issues can not be excluded. This report does not consist in a guarantee that no undeclared issues remain, nor should it be interpreted as such.